

СТРАТЕГІЯ ЗАЛУЧЕННЯ ТА КОМУНІКАЦІЇ (SOCIAL MEDIA & SEEDING PLAN)

Мета: Валідація ігрового MVP та залучення перших 100 активних користувачів через комбінований "посів" у технічних та ігрових спільнотах.

А) Вибір каналів комунікації (Channel Selection)

Стратегія дистрибуції базується на подвійному позиціонуванні проекту: як інноваційного інженерного рішення та як хардкорного ігрового продукту.

Цільовий сегмент	Пріоритетні платформи	Обґрунтування вибору (Стратегія)	Очікуваний результат
Технічна спільнота (Dev/Tech)	Reddit (r/godot, r/gamedev), DOU, GitHub	Build in Public: Демонстрація складної клієнт-серверної архітектури (Godot + NestJS) залучає інженерів, які готові тестувати код, шукати вразливості	Бета-тестери з технічним бекграундом, GitHub Stars, аудит безпеки бекенду.

		в API та давати високоякісний фідбек.	
Гейм-ентузіасти (Mass Market)	X (Twitter), YouTube Shorts	Visual Showcase: Жанр Action-RPG з елементами Extraction Shooter найкраще продається через візуальну динаміку. Демонстрація сортування Grid-інвентарю та хвильового спавну створює віральний ефект.	Формування ядра гравців для перевірки навантаження на PostgreSQL та метрик утримання (Retention).

Б) Стратегія запуску на Product Hunt (Launch Algorithm)

Product Hunt використовується для отримання міжнародного фідбеку та первинного трафіку на репозиторій проекту.

1. Hunter Kit (Базовий маркетинговий набір)

- Tagline (58 символів): Server-Auth Extraction Shooter built with Godot & NestJS.

- Description (229 символів): Maditron is a top-down tactical shooter featuring a 2D grid inventory. Powered by Godot and a NestJS/PostgreSQL backend, it guarantees secure, cheat-free 15-minute survival runs with strict server-side action validation.
- Медіа-активи (Media Assets):
 - Логотип: Анімований GIF з гліч-ефектом назви гри (передає атмосферу неоновому кіберпанку/хардкору).
 - Галерея 1: Скріншот роботи Grid-інвентарю (демонстрація стакування предметів та лічильників).
 - Галерея 2: Геймплейний момент (робота динамічного спавнера, знищення ворогів).
 - Галерея 3: Інженерна схема (Data Flow) від клієнта Godot до бази даних PostgreSQL.

2. Перший коментар (Maker's Comment)

Привіт, спільното Product Hunt!

Ми команда інженерів, яка втомилася від того, що в більшості інді-ігор прогрес легко зламати через редагування локальних збережень. Сьогодні ми представляємо Maditron спробу принести enterprise-рівень безпеки в жанр хардкорних top-down шутерів.

Ми хотіли створити гру, де кожен знайдений предмет має реальну вагу, а таблиці лідерів відображають справжній скіл, а не вміння користуватися Cheat Engine.

Наші ключові технічні рішення:

- Ігровий клієнт (Godot): Плавний UI та тактильний Grid-інвентар, побудовані на системі сигналів (GDScript).

- Server Authority (NestJS): Авторитарний бекенд, який жорстко валідує підбір луту, стакування предметів та розрахунків НР.
- Надійність (PostgreSQL): Транзакційна цілісність даних ігрових сесій.

Ми публікуємо проєкт з відкритою архітектурою. Будемо раді відповісти на ваші запитання щодо синхронізації клієнта та сервера. Що, на вашу думку, нам варто оптимізувати в першу чергу?

В) План публікацій (Content Plan)

Публікація 1: Емоційний фокус (Біль – Рішення – Результат)

Платформа: X (Twitter), профільні спільноти гравців.

Медіа: 15-секундне відео динамічної перестрілки та швидкого перетягування аптечки в інвентарі.

Текст публікації:

Скільки разів ви втрачали інтерес до гри після того, як бачили в таблиці лідерів гравця з неможливими показниками, досягнутими за 5 хвилин? Ми знаємо цей біль. Відсутність серверного захисту вбиває змагальний дух у 90% інді-шутерів.

Ми створили Maditron, щоб змінити цей стандарт.

Більше жодних маніпуляцій файлами. Кожна куля у вашому магазині та кожна аптечка у вашому рюкзаку валідуються на нашому виділеному сервері.

Тактична глибина. Мікроменеджмент двовимірного Grid-інвентарю, де правильне розміщення предметів вирішує результат бою.

Динаміка. Швидкі 15-хвилинні сесії з розумним спавном ворогів.

Ми створили надійну платформу для тих, хто цінує чесний хардкор. Сервери вже запущені, приєднуйтеся до закритого бета-тесту.

Долучитися до серверів: [Посилання]

#IndieGame #GodotEngine #Gamer #ExtractionShooter #GameDev

Публікація 2: Технічний сторітелінг (Build in Public)

Платформа: DOU, Reddit (r/gamedev), LinkedIn.

Медіа: Карусель зображень: Фрагмент коду на GDScript (відправка HTTP-запиту) + Фрагмент коду NestJS (контролер валідації інвентарю).

Текст публікації:

Як ми реалізували миттєвий Grid-інвентар у Godot із повною валідацією на NestJS без мережевих фризів.

Під час розробки нашого шутера Maditron ми зіткнулися з класичним архітектурним конфліктом: як забезпечити Server Authority (захист від читів), не перетворивши динамічну гру на покрокову стратегію через мережевий ping.

Коли гравець підіймає предмет або переміщує його в інший слот, клієнт не може сліпо довіряти цій дії. Але чекати відповіді

сервера для відмальовки UI - це гарантований злам ігрового потоку (UX).

Як ми вирішили цю проблему:

- Крок 1: Optimistic UI у Godot. Ми налаштували систему сигналів (Signals) так, що візуальне переміщення предмета в сітці відбувається миттєво локально.
- Крок 2: Асинхронна черга. Одночасно клієнт формує легкий HTTP-запит до нашого бекенду на NestJS у фоновому потоці.
- Крок 3: Серверні транзакції. NestJS перевіряє ліміти ваги/розміру та оновлює стани (наприклад, стакування набоїв) у PostgreSQL через атомарні транзакції. У разі розсинхронізації або нелегітимної дії сервер надсилає сигнал "Rollback", і клієнт плавно повертає предмет на початкове місце.

Результат: Гравці отримують ідеальну тактильну віддачу (час реакції UI < 16 мс), а база даних гарантує абсолютну неможливість дюпання (duplication) предметів.

Ми відкрили репозиторій для архітектурного рев'ю. Запрошуємо бекенд-інженерів поділитися думками щодо оптимізації таких транзакцій під високим навантаженням!

Архітектура та код тут: [Посилання на GitHub]

Документація API (Swagger): [Посилання]

#SoftwareEngineering #Godot #NestJS #SystemDesign #Backend